

COMP4131: Data Modelling and Analysis

Lecture 7: Gaussian Process Regression and Classification

Daokun Zhang

University of Nottingham Ningbo China

daokun.zhang@nottingham.edu.cn

1 April, 2025

- 1 Gaussian Process
- 2 Gaussian Process Regression
- 3 Gaussian Process Classification

Gaussian Process 高斯过程

Linear Regression

Recall the probabilistic analysis of the standard linear regression model with Gaussian noise

$$f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}), \quad y = f(\mathbf{x}) + \varepsilon,$$

where ε is the additive noise that follows an i.i.d Gaussian distribution with zero mean and variance σ_n^2 :

$$\varepsilon \sim \mathcal{N}(0, \sigma_n^2).$$

Given the training data $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and $\mathbf{y} = \{y_1, \dots, y_N\}$, we can use maximum likelihood estimation (MLE) to find the solution of $\mathbf{w}$ as

$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y},$$

where $\Phi = [\phi(\mathbf{x}_1), \dots, \phi(\mathbf{x}_N)]^T$ is an $N \times M$ matrix.

Bayesian Linear Regression

In the Bayesian formalism, we can specify a prior over the parameters, expressing our beliefs about the parameters before we look at the observations. We put a Gaussian prior with mean vector $\mu_{\mathbf{w}}$ and covariance matrix $\Sigma_{\mathbf{w}}$ on the weights

$$\mathbf{w} \sim \mathcal{N}(\mu_{\mathbf{w}}, \Sigma_{\mathbf{w}}).$$

Due to the randomness in $\mathbf{w}$, $f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$ is no longer a deterministic but a **random function**. Let f_p denote the linear function value $f(\mathbf{x}_p)$ at sample $\mathbf{x}_p$, then $f_1, \dots, f_N$ follow a multivariate Gaussian distribution with

$$\mathbb{E}[f_p] = \mu_{\mathbf{w}}^T \phi(\mathbf{x}_p),$$

$$\text{Cov}(f_p, f_q) = \phi(\mathbf{x}_p)^T \Sigma_{\mathbf{w}} \phi(\mathbf{x}_q).$$

Now we get a **Gaussian process** $f(\mathbf{x})$ with mean function $m(\mathbf{x}) = \mu_{\mathbf{w}}^T \phi(\mathbf{x})$ and covariance function $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^T \Sigma_{\mathbf{w}} \phi(\mathbf{x}')$.

Definition: *A Gaussian process is a collection of random variables, any finite number of which have a joint Gaussian distribution.*

高斯过程是一组随机变量，其中任意有限数量的变量都具有联合高斯分布。总结：任意有限个随机变量都服从联合高斯分布

Implications

- “a collection of random variables” means a set that has
 - 一组随机变量 ”指的是一个集合，该集合包含
 - a finite number of random variables 有限数量的随机变量
 - an infinite number of random variables 无限数量的随机变量
- “any finite number of which have a joint Gaussian distribution” means: 其中任何有限数量的随机变量都具有联合高斯分布
 - multiple random variables follow a joint multivariate Gaussian distribution 多个随机变量遵循联合多元高斯分布
 - after marginalization, a single random variable follows a univariate Gaussian distribution 经过边缘化处理后，单个随机变量遵循单变量高斯分布

Gaussian Process

A Gaussian process is completely specified by its **mean function** and **covariance function**. We define **mean function** $m(\mathbf{x})$ and the **covariance function** $k(\mathbf{x}, \mathbf{x}')$ as

$$m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})],$$

$$k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[\{f(\mathbf{x}) - m(\mathbf{x})\} \{f(\mathbf{x}') - m(\mathbf{x}')\}],$$

and write the Gaussian process as

$$f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')).$$

Usually, for notational simplicity we will **take the mean function to be zero**, i.e., set $m(\mathbf{x}) = 0$.

Example: For **Bayesian linear regression model** $f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x})$ with prior $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{w}})$, we have the mean and covariance function

$$m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})] = \phi(\mathbf{x})^T \mathbb{E}[\mathbf{w}] = 0,$$

$$k(\mathbf{x}, \mathbf{x}') = \mathbb{E}[f(\mathbf{x})f(\mathbf{x}')] = \phi(\mathbf{x})^T \mathbb{E}[\mathbf{w}\mathbf{w}^T] \phi(\mathbf{x}') = \phi(\mathbf{x})^T \Sigma_{\mathbf{w}} \phi(\mathbf{x}').$$

Kernel Trick

The covariance function $k(\mathbf{x}, \mathbf{x}')$ can be extended to any **kernel functions**.

For all $\mathbf{x}$ and $\mathbf{x}'$ in the input space $\mathcal{X}$, the function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is called a kernel function, if there exists a feature map $\varphi : \mathcal{X} \rightarrow \mathcal{V}$ that satisfies

$$k(\mathbf{x}, \mathbf{x}') = \langle \varphi(\mathbf{x}), \varphi(\mathbf{x}') \rangle_{\mathcal{V}},$$

where $\mathcal{V}$ is a inner product space, and $\langle \cdot, \cdot \rangle_{\mathcal{V}}$ is the inner product operation defined by $\mathcal{V}$.

An alternative definition can be formulated by the **positive semidefinite (PSD)** property: For any points $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ in $\mathcal{X}$, and all choices of n real-valued coefficients $(c_1, \dots, c_n)$, the function $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is a kernel function, if

$$\sum_{i=1}^n \sum_{j=1}^n k(\mathbf{x}_i, \mathbf{x}_j) c_i c_j \geq 0.$$

The spanned matrix, $\mathbf{K} \in \mathbb{R}^n \times \mathbb{R}^n$ with its ij -th entry $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$, is called **Gram matrix**.

Many kernels can be chosen for various application scenarios

- Fisher kernel
- Polynomial kernel
- Radial basis function kernel (RBF)
- String kernels
- Graph kernels

费希尔核
多项式核
径向基函数核 (RBF)
字符串核
图核

The Radial basis function kernel (RBF) is also called squared-exp or Gaussian kernel, which is formulated as

$$k_{\text{SE}}(\mathbf{x}_i, \mathbf{x}_j) = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\ell^2}\right).$$

Gaussian Process Regression

Prediction with Noise-free Observations

We first consider the ideal case where the observation y can be described by a Gaussian process $f(\mathbf{x}) \sim \mathcal{GP}(0, k(\mathbf{x}, \mathbf{x}'))$ in a **noise-free** manner, i.e., $y = f(x)$. For a set of test samples with size n_* , and feature matrix $\mathbf{X}_*$, its function value vector $\mathbf{f}_*$ follows a multivariate Gaussian distribution

$$\mathbf{f}_* \sim \mathcal{N}(\mathbf{0}, \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*)),$$

where $\mathbf{K}(\cdot, \cdot)$ is the covariance matrix between two collections of input feature vectors.

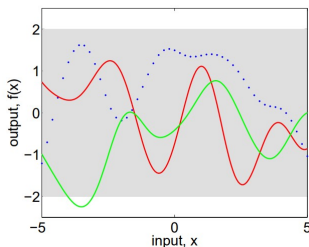
If we are given a set of training samples with size n , and feature matrix $\mathbf{X}$, its function value vector $\mathbf{f}$ should follow a multivariate Gaussian distribution jointly with $\mathbf{f}_*$

$$\begin{bmatrix} \mathbf{f} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) & \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\ \mathbf{K}(\mathbf{X}_*, \mathbf{X}) & \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \end{bmatrix}\right).$$

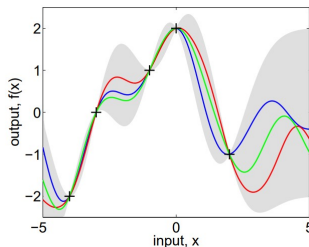
Prediction with Noise-free Observations

Given the observations of $\mathbf{f}$, the **posterior distribution** of $\mathbf{f}_*$ is also a Gaussian distribution

$$\mathbf{f}_* | \mathbf{X}_*, \mathbf{X}, \mathbf{f} \sim \mathcal{N} \left(\mathbf{K}(\mathbf{X}_*, \mathbf{X}) \mathbf{K}(\mathbf{X}, \mathbf{X})^{-1} \mathbf{f} \right. \\ \left. \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) - \mathbf{K}(\mathbf{X}_*, \mathbf{X}) \mathbf{K}(\mathbf{X}, \mathbf{X})^{-1} \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \right).$$



(a), prior



(b), posterior

The prior and posterior distributions of $\mathbf{f}_*$ with the RBF kernel function $k(\mathbf{x}_p, \mathbf{x}_q) = \exp(-\frac{1}{2}\|\mathbf{x}_p - \mathbf{x}_q\|_2^2)$.

Prediction using Noisy Observations

It is typical for more realistic modelling situations that we do not have access to function values themselves, but only **noisy versions** thereof $y = f(\mathbf{x}) + \varepsilon$.

Assuming additive **independent identically distributed** Gaussian noise ε with zero mean and variance σ_n^2 , the prior on the noisy observations becomes another Gaussian distribution with mean $\mathbb{E}[y] = \mathbb{E}[f(\mathbf{x}) + \varepsilon] = 0$, and covariance

$$\text{cov}(y_p, y_q) = k(\mathbf{x}_p, \mathbf{x}_q) + \sigma_n^2 \delta_{pq} \text{ or } \text{cov}(\mathbf{y}) = K(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I},$$

where δ_{pq} is a Kronecker delta symbol which is one if and only if $p = q$ and zero otherwise.

Gaussian Process Regression

We can write the joint distribution of the observed target values $\mathbf{y}$ and the function values $\mathbf{f}_*$ of test samples under the prior as

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_* \end{bmatrix} \sim \mathcal{N} \left(\mathbf{0}, \begin{bmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I} & \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\ \mathbf{K}(\mathbf{X}_*, \mathbf{X}) & \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \end{bmatrix} \right).$$

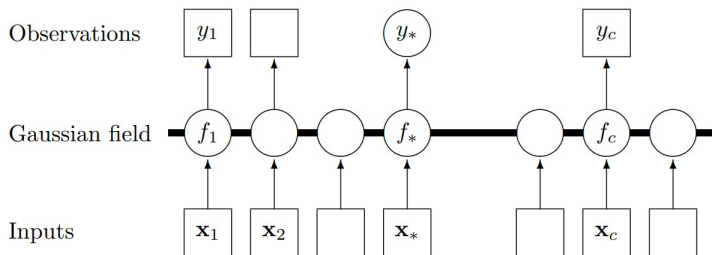
Deriving the conditional distribution, we arrive at the [key predictive equations](#) for [Gaussian process regression](#)

$\mathbf{f}_* | \mathbf{X}, \mathbf{y}, \mathbf{X}_* \sim \mathcal{N}(\bar{\mathbf{f}}_*, \text{cov}(\mathbf{f}_*))$, where

$$\bar{\mathbf{f}}_* \triangleq \mathbb{E}[\mathbf{f}_* | \mathbf{X}, \mathbf{y}, \mathbf{X}_*] = \mathbf{K}(\mathbf{X}_*, \mathbf{X}) [\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I}]^{-1} \mathbf{y},$$

$$\text{cov}(\mathbf{f}_*) = \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) - \mathbf{K}(\mathbf{X}_*, \mathbf{X}) [\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma_n^2 \mathbf{I}]^{-1} \mathbf{K}(\mathbf{X}, \mathbf{X}_*).$$

Gaussian Process Regression



Graphical model (chain graph) for a Gaussian process for regression. Squares represent **observed variables** and circles represent **unknowns**.

Gaussian Process Regression

By using a compact form of the notation setting $\mathbf{K} = \mathbf{K}(\mathbf{X}, \mathbf{X})$ and $\mathbf{k}_* = \mathbf{K}(\mathbf{X}, \mathbf{x}_*)$ for a test sample $\mathbf{x}_*$, the prediction f_* for the single test sample $\mathbf{x}_*$ has mean $\bar{f}_*$ and variance $\mathbb{V}[f_*]$ as

$$\bar{f}_* = \mathbf{k}_*^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y},$$
$$\mathbb{V}[f_*] = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^T (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{k}_*.$$

Another way to look at the expression of the mean $\bar{f}_*$ is to see it as a **linear combination** of n kernel functions, each one centered on a training point, by writing

$$\bar{f}_* = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{x}_*),$$

where $\boldsymbol{\alpha} = (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}$.

Gaussian Process Regression

input: X (inputs), $\mathbf{y}$ (targets), k (covariance function), σ_n^2 (noise level), $\mathbf{x}_*$ (test input)

2: $L := \text{cholesky}(K + \sigma_n^2 I)$
 $\boldsymbol{\alpha} := L^\top \backslash (L \backslash \mathbf{y})$

4: $\bar{f}_* := \mathbf{k}_*^\top \boldsymbol{\alpha}$ } predictive mean
 $\mathbf{v} := L \backslash \mathbf{k}_*$ }

6: $\mathbb{V}[f_*] := k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{v}^\top \mathbf{v}$ } predictive variance
 $\log p(\mathbf{y}|X) := -\frac{1}{2} \mathbf{y}^\top \boldsymbol{\alpha} - \sum_i \log L_{ii} - \frac{n}{2} \log 2\pi$

8: **return:** $\bar{f}_*$ (mean), $\mathbb{V}[f_*]$ (variance), $\log p(\mathbf{y}|X)$ (log marginal likelihood)

Algorithm for Gaussian process regression, where the predictions (the target value's mean and variance) at the test sample $\mathbf{x}_*$ and the [log marginal likelihood](#) on training set are returned.

Varying the Hyperparameters

Setting the RBF kernel $k(\mathbf{x}_p, \mathbf{x}_q) = \sigma_f^2 \exp(-\frac{1}{2\ell^2} \|\mathbf{x}_p - \mathbf{x}_q\|_2^2)$ as the covariance function, the covariance between the observations y_p and y_q becomes

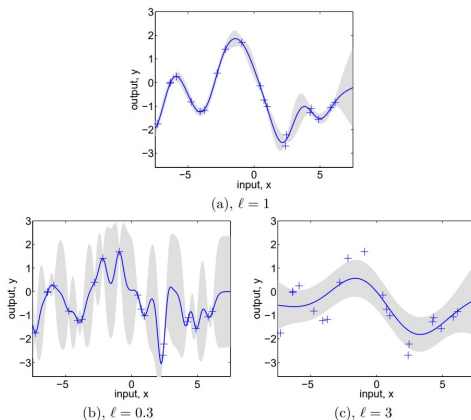
$$\text{cov}(y_p, y_q) = \sigma_f^2 \exp(-\frac{1}{2\ell^2} \|\mathbf{x}_p - \mathbf{x}_q\|_2^2) + \sigma_n^2 \delta_{pq},$$

which implies that the model relies on three hyperparameters:

- signal variance σ_f^2
- noise variance σ_n^2
- length-scale ℓ

The [log marginal likelihood](#) can be used to tune the hyperparameters.

Varying the Hyperparameters



Small length-scale $\ell = 0.3 \rightarrow$ a sharply varying function f with small noise.

Large length-scale $\ell = 3 \rightarrow$ a slowly varying function f with large noise.

Moderate length-scale $\ell = 1 \rightarrow$ exact fitting of the underlying function f .

Gaussian Process Classification

Binary Gaussian Process Classifier

For binary classification with label candidate set $\{+1, -1\}$, logistic regression uses the **sigmoid** function to squash the linear regression output into the label probability $p(y = +1|\mathbf{x}) = \sigma(\mathbf{w}^T \phi(\mathbf{x}))$.

In Gaussian process regression, the latent function value $f(\mathbf{x})$ is used to directly construct the regression output.

For **Gaussian process classification**, we can also construct the **label probability** via the **sigmoid** function

$$p(y = +1|f(\mathbf{x})) = \sigma(f(\mathbf{x})).$$

Different from $\mathbf{w}^T \phi(\mathbf{x})$, $f(\mathbf{x})$ is Gaussian random variable, we need to marginalize it to get the posterior probability

$$p(y = +1|\mathbf{x}) = \int p(y = +1|f)p(f|\mathbf{x})df.$$

Binary Gaussian Process Classifier

Given a training set $\{\mathbf{X}, \mathbf{y}\}$, for a new test sample $\mathbf{x}_*$, Gaussian process classifier predicts its label by estimating the posterior probability

$$\begin{aligned}\bar{\pi}_* &\triangleq p(y_* = +1 | \mathbf{X}, \mathbf{y}, \mathbf{x}_*) = \int p(y_* = +1 | f_*) p(f_* | \mathbf{X}, \mathbf{y}, \mathbf{x}_*) df_* \\ &= \int \sigma(f_*) p(f_* | \mathbf{X}, \mathbf{y}, \mathbf{x}_*) df_*.\end{aligned}$$

To complete the estimation, we need first calculate the posterior distribution of the latent variable f_* conditioned on the test sample $\mathbf{x}_*$

$$p(f_* | \mathbf{X}, \mathbf{y}, \mathbf{x}_*) = \int p(f_* | \mathbf{X}, \mathbf{x}_*, \mathbf{f}) p(\mathbf{f} | \mathbf{X}, \mathbf{y}) d\mathbf{f},$$

where $p(\mathbf{f} | \mathbf{X}, \mathbf{y}) = p(\mathbf{y} | \mathbf{f}) p(\mathbf{f} | \mathbf{X}) / p(\mathbf{y} | \mathbf{X})$ is the posterior over the latent variables and $p(\mathbf{y} | \mathbf{f})$ is computed by applying the elementwise sigmoid function to $\mathbf{f}$.

Laplace Approximation for Binary GP Classifier

In the regression case, $\mathbf{y}$ and f_* jointly follow a multivariate Gaussian distribution, the conditional probability distribution $p(f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*)$ can be computed analytically.

However, in the classification case, due to the computational intractability of the distribution $p(\mathbf{f}|\mathbf{X}, \mathbf{y})$, the posterior distribution $p(f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*)$ cannot be computed analytically with the intractable integral.

To solve this problem, we resort to the [Laplace approximation](#) that approximates the distribution $p(\mathbf{f}|\mathbf{X}, \mathbf{y})$ with a Gaussian distribution $q(\mathbf{f}|\mathbf{X}, \mathbf{y})$, obtained by doing a second-order Taylor expansion of $\log p(\mathbf{f}|\mathbf{X}, \mathbf{y})$ around the maximum of the posterior

$$q(\mathbf{f}|\mathbf{X}, \mathbf{y}) = \mathcal{N}(\mathbf{f}|\hat{\mathbf{f}}, \mathbf{A}^{-1}) \propto \exp\left(-\frac{1}{2}(\mathbf{f} - \hat{\mathbf{f}})^T \mathbf{A}(\mathbf{f} - \hat{\mathbf{f}})\right),$$

where $\hat{\mathbf{f}} = \operatorname{argmax}_{\mathbf{f}} p(\mathbf{f}|\mathbf{X}, \mathbf{y})$ and $\mathbf{A} = -\nabla\nabla \log p(\mathbf{f}|\mathbf{X}, \mathbf{y})|_{\mathbf{f}=\hat{\mathbf{f}}}$ is the Hessian matrix of the negative log posterior at that point.

Laplace Approximation for Binary GP Classifier

By Bayes' rule, the posterior over the latent variables $p(\mathbf{f}|\mathbf{X}, \mathbf{y})$ is given by

$$p(\mathbf{f}|\mathbf{X}, \mathbf{y}) = \frac{p(\mathbf{y}, \mathbf{f}|\mathbf{X})}{p(\mathbf{y}|\mathbf{X})} = \frac{p(\mathbf{y}|\mathbf{f})p(\mathbf{f}|\mathbf{X})}{p(\mathbf{y}|\mathbf{X})},$$

but $p(\mathbf{y}|\mathbf{X})$ is independent of $\mathbf{f}$, we only need to consider the un-normalized posterior $p(\mathbf{y}, \mathbf{f}|\mathbf{X}) = p(\mathbf{y}|\mathbf{f})p(\mathbf{f}|\mathbf{X})$ when maximizing w.r.t. $\mathbf{f}$.

Taking the logarithm on the un-normalized posterior, we get

$$\begin{aligned}\Psi(\mathbf{f}) &\triangleq \log p(\mathbf{y}|\mathbf{f}) + \log p(\mathbf{f}|\mathbf{X}) \\ &= \log p(\mathbf{y}|\mathbf{f}) - \frac{1}{2}\mathbf{f}^T \mathbf{K}^{-1}\mathbf{f} - \frac{1}{2}\log |\mathbf{K}| - \frac{n}{2}\log 2\pi,\end{aligned}$$

where $p(\mathbf{f}|\mathbf{X})$ is a Gaussian distribution $\mathcal{N}(\mathbf{0}, \mathbf{K})$, $\mathbf{K}$ is short for the kernel matrix $\mathbf{K}(\mathbf{X}, \mathbf{X})$, and $|\mathbf{K}|$ is the determinant of the kernel matrix $\mathbf{K}$.

Laplace Approximation for Binary GP Classifier

Differentiating $\Psi(\mathbf{f})$ w.r.t. $\mathbf{f}$, we obtain

$$\begin{aligned}\nabla \Psi(\mathbf{f}) &= \nabla \log p(\mathbf{y}|\mathbf{f}) - \mathbf{K}^{-1}\mathbf{f}, \\ \nabla \nabla \Psi(\mathbf{f}) &= \nabla \nabla \log p(\mathbf{y}|\mathbf{f}) - \mathbf{K}^{-1},\end{aligned}$$

where $\nabla \nabla \log p(\mathbf{y}|\mathbf{f})$ is diagonal, since the distribution for $y_i = +1$ or -1 depends only on f_i , not on $f_{j \neq i}$.

We can apply the [Newton-Raphson](#) method to find the latent function value vector $\hat{\mathbf{f}}$ that maximizes $\Psi(\mathbf{f})$, and obtain the approximated probability distribution as

$$q(\mathbf{f}|\mathbf{X}, \mathbf{y}) = \mathcal{N}\left(\hat{\mathbf{f}}, (\mathbf{K}^{-1} + \mathbf{W})^{-1}\right),$$

where $\mathbf{W} \triangleq -\nabla \nabla \log p(\mathbf{y}|\mathbf{f})|_{\mathbf{f}=\hat{\mathbf{f}}}$.

Laplace Approximation for Binary GP Classifier

With $q(\mathbf{f}|\mathbf{X}, \mathbf{y})$ approximating $p(\mathbf{f}|\mathbf{X}, \mathbf{y})$, we can get the approximation for the posterior $p(f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*)$ as

$$q(f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*) = \int p(f_*|\mathbf{X}, \mathbf{x}_*, \mathbf{f}) q(\mathbf{f}|\mathbf{X}, \mathbf{y}) d\mathbf{f},$$

where both $p(f_*|\mathbf{X}, \mathbf{x}_*, \mathbf{f})$ and $q(\mathbf{f}|\mathbf{X}, \mathbf{y})$ are Gaussian distributions. It can be proved that $q(f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*)$ is also a Gaussian distribution with mean and variance as

$$\mathbb{E}_q[f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*] = \mathbf{k}_*^T \mathbf{K}^{-1} \hat{\mathbf{f}},$$

$$\mathbb{V}_q[f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*] = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}_*^T (\mathbf{K} + \mathbf{W}^{-1})^{-1} \mathbf{k}_*.$$

Finally, the label probability for the test sample $\mathbf{x}_*$ can be computed as

$$\bar{\pi}_* \triangleq p(y_* = +1|\mathbf{X}, \mathbf{y}, \mathbf{x}_*) := \int \sigma(f_*) q(f_*|\mathbf{X}, \mathbf{y}, \mathbf{x}_*) df_*.$$

Numeric integration techniques are generally used to calculate the one-dimensional integral.

Binary Gaussian Process Classifier

```
input:  $K$  (covariance matrix),  $\mathbf{y}$  ( $\pm 1$  targets),  $p(\mathbf{y}|\mathbf{f})$  (likelihood function)
2:  $\mathbf{f} := \mathbf{0}$  initialization
   repeat Newton iteration
4:    $W := -\nabla \nabla \log p(\mathbf{y}|\mathbf{f})$ 
      $L := \text{cholesky}(I + W^{\frac{1}{2}} K W^{\frac{1}{2}})$ 
6:    $\mathbf{b} := W\mathbf{f} + \nabla \log p(\mathbf{y}|\mathbf{f})$ 
      $\mathbf{a} := \mathbf{b} - W^{\frac{1}{2}} L^{\top} \setminus (L \setminus (W^{\frac{1}{2}} K \mathbf{b}))$ 
8:    $\mathbf{f} := K\mathbf{a}$ 
   until convergence
10:  $\log q(\mathbf{y}|\hat{\mathbf{X}}, \theta) := -\frac{1}{2} \mathbf{a}^{\top} \mathbf{f} + \log p(\mathbf{y}|\mathbf{f}) - \sum_i \log L_{ii}$ 
   return:  $\hat{\mathbf{f}} := \mathbf{f}$  (post. mode),  $\log q(\mathbf{y}|\hat{\mathbf{X}}, \theta)$  (approx. log marg. likelihood)
```

Training algorithm for the binary Gaussian process classifier with Laplace approximation. The mode $\hat{\mathbf{f}}$ for the posterior probability distribution $p(\mathbf{f}|\mathbf{X}, \mathbf{y})$ is returned, as well as the **approximated log marginal likelihood**, which can be use to tune hyperparameters.

Binary Gaussian Process Classifier

input: $\hat{\mathbf{f}}$ (mode), X (inputs), $\mathbf{y}$ (± 1 targets), k (covariance function),
 $p(\mathbf{y}|\mathbf{f})$ (likelihood function), $\mathbf{x}_*$ test input

2: $W := -\nabla \nabla \log p(\mathbf{y}|\hat{\mathbf{f}})$

$L := \text{cholesky}(I + W^{\frac{1}{2}} K W^{\frac{1}{2}})$

4: $\bar{f}_* := \mathbf{k}(\mathbf{x}_*)^\top \nabla \log p(\mathbf{y}|\hat{\mathbf{f}})$

$\mathbf{v} := L \backslash (W^{\frac{1}{2}} \mathbf{k}(\mathbf{x}_*))$

6: $\mathbb{V}[f_*] := k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{v}^\top \mathbf{v}$

$\bar{\pi}_* := \int \sigma(z) \mathcal{N}(z|\bar{f}_*, \mathbb{V}[f_*]) dz$

8: **return:** $\bar{\pi}_*$ (predictive class probability (for class 1))

Prediction algorithm for the binary Gaussian classifier with Laplace approximation.

Multi-Class Gaussian Process Classifier

For the multi-class classification problem, where the class label y belongs to the label candidate set $\{1, \dots, C\}$ with size C , we need to construct a Gaussian process for each class $c \in \{1, \dots, C\}$, with the latent function value $f^c(\mathbf{x})$ evaluated at $\mathbf{x}$.

The conditional label probability $p(y = c | f^1(\mathbf{x}), \dots, f^C(\mathbf{x}))$ can be constructed by the softmax function

$$p(y = c | f^1(\mathbf{x}), \dots, f^C(\mathbf{x})) = \frac{\exp(f^c(\mathbf{x}))}{\sum_{c'=1}^C \exp(f^{c'}(\mathbf{x}))}.$$

Following the same procedure as the binary classification case, given a test sample $\mathbf{x}_*$, we can estimate the posterior probabilities $p(y_* = c | \mathbf{X}, \mathbf{y}, \mathbf{x}_*)$ to predict $\mathbf{x}_*$'s class label.

Williams CK, Rasmussen CE. **Gaussian Processes for Machine Learning**. Cambridge, MA: MIT press; 2006.

- Section 1 - Section 3

The End